

1 Introduction

This Hello World! chapter provides an initial introduction to the use of artificial intelligence (AI) tools in program development. With examples, the chapter shows the use of chat-based tools such as ChatGPT and coding wizards such as GitHub Copilot.

You've probably already tried out AI coding tools. Nevertheless, you should skim the following pages to get some new ideas in the introductory chapter before we go into more depth in the later chapters. For example, [Section 1.5](#) contains a concise summary of how language models work behind the scenes. This knowledge will help you better understand the limits of current technology.

1.1 Coding via Chat

Chat systems such as ChatGPT provide the most intuitive introduction to AI-supported coding. When we wrote this book, there were various free systems to choose from:

- ChatGPT (the original from OpenAI; <https://chatgpt.com>)
- Copilot (Microsoft using ChatGPT/OpenAI; <https://copilot.microsoft.com>)
- Gemini (Google; <https://gemini.google.com>)
- Claude (Anthropic; <https://claude.ai>)
- DeepSeek (the disrupting newcomer from China; <https://chat.deepseek.com>)

On most websites, you have the option to activate additional functions by purchasing a paid subscription. To the extent that we experimented with publicly available chat systems while working on this book, ChatGPT was our preferred system. You must register once on this website, but that's

quick and doesn't cost anything. Then, the application is very simple: you formulate your question and receive an answer.

1.1.1 Password Generator

You need a function in a Python script that returns a random password of a specified length. Of course, you can write the code yourself, but you'll probably save yourself a minute or two if you hand over this trivial task to ChatGPT.



Figure 1.1 ChatGPT Wrapping a Password Generator into a Function

You can now easily copy and paste the code into your script. Don't forget the necessary imports! These are always specified at the beginning of the script.

1.1.2 Python: Searching and Replacing Dates

In this example, all dates in a character string are supposed to be converted from the US format (12/31/2024) to the ISO format (2024-12-31). The programming language to be used is Python.

The code uses the `re` module, which is used to process regular expressions in Python. ChatGPT formulates the code as a function, which simplifies fur-

ther use. `convert_us_to_iso_date` in turn contains a subfunction named `replace`, which gets passed as a parameter to `re.sub` and replaces each hit with the newly formatted date.

Prompt: Python: I want to replace all US dates by ISO dates in a string. Can you help?

Certainly! To replace all US date formats (commonly MM/DD/YYYY) with ISO date formats (YYYY-MM-DD) in a string using Python, you can use the `re` module to find dates and then rearrange the format. Here's a Python function that accomplishes this:

```
import re

def convert_us_to_iso_dates(text):
    # Pattern to find dates in MM/DD/YYYY format
    date_pattern = r'\b(\d{1,2})/(\d{1,2})/(\d{4})\b'

    # Function to replace each match
    def replace(match):
        month, day, year = match.groups()
        return f'{year}-{month.zfill(2)}-{day.zfill(2)}'

    # Replace all matches in the text
    return re.sub(date_pattern, replace, text)

# Example usage
input_text = "Today's date is 05/09/2024 and the event"
" +
            "was on 12/25/2023."
converted_text = convert_us_to_iso_dates(input_text)
print(converted_text)
```

This script:

- Defines a regex pattern to match dates in the format MM/DD/YYYY.
- Uses a replacement function to rearrange the date components into the ISO format YYYY-MM-DD.
- Applies this transformation to all occurrences in the provided text.

The code works perfectly in the form presented here. If you're not familiar with regular expressions, you can have the structure of the pattern explained to you in a follow-up question:

Prompt: Could you explain the regular pattern?

Certainly! Here's a breakdown of the pattern

```
r'\b(\d{1,2})/(\d{1,2})/(\d{4})\b':
```

- `\b`: This is a word boundary marker. It ensures that the pattern matches whole words only, which helps prevent partial matches of larger sequences that aren't dates.
- `(\d{1,2})`: This part of the pattern captures one or two digits. The `\d` denotes a digit (equivalent to `[0-9]`), and `{1,2}` specifies that the digit can appear once or twice. This pattern is used for both the month and the day in the date.
- `/`: This is a literal forward slash character. It separates the month and day, and day and year, in US date formats.
- `(\d{4})`: This captures exactly four digits, representing the year. Here, `\d` again denotes a digit, and `{4}` specifies that exactly four digits must be present.
- `\b`: Another word boundary marker, ensuring the date is a standalone entity, not part of a larger sequence of numbers or characters.

So, the entire pattern matches strings that start and end with a boundary between words (like spaces or punctuation), followed by one or two digits for the month, a slash, one or two digits for the day, another slash, and finally four digits for the year.

In this book, we normally refrain from reproducing such explanations or abbreviate ChatGPT outputs if they are too verbose because we're primarily interested in how you can best formulate the prompt.

ChatGPT usually uses variable names composed with an underscore (`convert_us_to_iso_dates`) in Python code. If you prefer other conventions or if other rules apply in your project, you can specify these directly in the prompt or in a follow-up question:

Prompt: Could you please regenerate the code with camelCase names?

Sure! Here's the Python code updated to use camelCase naming conventions for the function and variable names:

```
def convertUsToIsoDates(text):  
    # Pattern to find dates in MM/DD/YYYY format  
    datePattern = r'\b(\d{1,2})/(\d{1,2})/(\d{4})\b'
```

AI Results Aren't Reproducible!

If you enter the prompts printed previously, you'll receive different answers even if you're using exactly the same ChatGPT version. Current AI tools produce new text or code each time, so the result might be better or worse.

1.1.3 SQL: Analysis of a Table

The starting point for the following example is a table in MySQL that contains value-added tax (VAT) rates (sales tax rates) for different countries. As the tax rates can change over time, the `validFrom` column indicates the date from which the value is valid. What does the `SELECT` query look like that returns the tax rate valid at a certain time for a certain country?

For ChatGPT to give the correct answer, you must provide sufficient context information. In particular, it's important which database management system (DBMS) you're using (in this case, MySQL) and what the exact structure of the table is.

Prompt: MySQL: I have a table to store VAT rates for different countries. This is the table definition:

```
CREATE TABLE vat (  
    id          INT NOT NULL AUTO_INCREMENT,  
    country     CHAR(2) NOT NULL,  
    rate        DOUBLE NOT NULL,  
    validFrom   DATE NOT NULL,
```

```
PRIMARY KEY (id)
)
```

I want to get the VAT rate for a country (i.e., 'de') at a given time ('2024-04-15'). What should the query look like?

To retrieve the VAT rate for a specific country at a given time from your table, you need to write a SQL query that checks for the most recent rate that has been set before or on the date you're interested in. This involves selecting the correct entry based on the country code and the validFrom date.

Here is the SQL query to find the VAT rate for Germany ('DE') as of April 15, 2024:

```
SELECT rate
FROM vat
WHERE country = 'DE'
      AND validFrom <= '2024-04-15'
ORDER BY validFrom DESC
LIMIT 1;
```

ChatGPT provides the correct answer. The program then explains the structure of the SQL command in detail (not printed here). If you don't use SQL on a daily basis, this is a great help in understanding the code.

Note that you must explicitly state which database system you're using for questions of this type. LIMIT 1 is a MySQL-specific extension of the SQL syntax. The same question for SQL Server provides a different answer with TOP instead of LIMIT.

Prompt: Would this also work with SQL Server?

The only change needed to make it fully compatible with SQL Server is to replace LIMIT 1 with TOP (1), which is the SQL Server syntax for limiting the number of rows returned. Here's the revised query for SQL Server:

```
SELECT TOP (1) rate
FROM vat
```

```
WHERE country = 'DE'  
    AND validFrom <= '2024-04-15'  
ORDER BY validFrom DESC;
```

This shows us how important it is to specify the context of the question.

1.1.4 PHP: Bearer Token Authentication

Say you're familiar with PHP and the concepts of REST application programming interfaces (APIs), but you don't know how to evaluate a bearer token in a PHP program. No problem, ChatGPT can help you out:

Prompt: PHP/REST: How can I extract a bearer token from a GET request?

In PHP, when handling RESTful API requests that use a bearer token for authentication, you typically extract the token from the Authorization header of the incoming GET request. Here's a step-by-step guide on how to do this:

- Access the authorization header: PHP provides access to request headers through the `$_SERVER` superglobal array. The Authorization header can be accessed using the key `HTTP_AUTHORIZATION` within this array.
- Extract the bearer token: The value of the Authorization header generally starts with the word "Bearer", followed by a space and then the token itself. You'll need to parse this string to extract the token.

Here's an example code snippet that demonstrates how to extract a bearer token from a GET request:

```
<?php  
// Function to get the bearer token from the  
// Authorization header  
function getBearerToken() {  
    if (isset($_SERVER['HTTP_AUTHORIZATION'])) {  
        // Extract the header value  
        $header = $_SERVER['HTTP_AUTHORIZATION'];  
  
        // Check if the header starts with "Bearer"  
        if (preg_match('/Bearer\s(\S+)/', $header,
```

```

$matches)) {
    // Return the token part
    return $matches[1];
}
}
return null; // Return null if no token was found
}
?>

```

Handle edge cases:

- **Header Not Present:** If the Authorization header is not present, your function should handle this gracefully, perhaps returning `null` or an appropriate error message.
- **Malformed Authorization Header:** If the header is present but doesn't properly include a bearer token (e.g., missing the "Bearer" prefix), you should also handle this case, possibly logging the issue or returning an error.

Always ensure that tokens are handled securely:

- Validate the token if necessary (e.g., check its format, expiration, and authenticity).
- Use HTTPS to protect the token in transit.
- Limit the exposure of sensitive token information in logs or error messages.

This approach will help you securely extract and handle a bearer token in a PHP-based RESTful API environment.

1.1.5 Porting a Bash Script for PowerShell

ChatGPT can do more than just generate new code. It can also search for errors in given code; convert code from one programming language to another; develop unit tests for a given function; and restructure, comment, and explain existing code.

The last introductory example is about porting a simple Bash script for PowerShell. The original script monitors the Markdown files in a directory.

When changes are saved in a file and the Markdown file is newer than the corresponding PDF file, the PDF document is recreated using the pandoc program. This automated build process is now to be implemented on a Windows computer using PowerShell. That's no problem for ChatGPT!

Prompt: Can you provide a PowerShell equivalent to the following Bash script?

```
#!/bin/bash
while true; do
  for mdfile in *.md; do
    pdffile=${mdfile%.md}.pdf
    if [ $mdfile -nt $pdffile ]; then
      echo $mdfile
      pandoc ... $mdfile -o $pdffile
    fi
  done
  sleep 1
done
```

Certainly! Here is the PowerShell equivalent of the given Bash script. This script loops continuously, checking for .md files in the current directory. If an .md file is newer than its corresponding .pdf file, it uses Pandoc to convert the Markdown file to a PDF. It also waits for 1 second after each iteration.

```
while ($true) {
  Get-ChildItem -Filter *.md | ForEach-Object {
    $mdFile = $_
    $pdfFile = "$($mdFile.BaseName).pdf"

    if (-Not (Test-Path $pdfFile) -or `
        (Get-Item $mdFile).LastWriteTime -gt `
        (Get-Item $pdfFile).LastWriteTime)
    {
      Write-Output $mdFile
      pandoc $mdFile -o $pdfFile
    }
  }
}
```

```
}    Start-Sleep -Seconds 1
```

Make sure to customize any Pandoc-specific parameters according to your conversion requirements.

ChatGPT “understands” the task of the original script, even though it doesn’t contain any comments. The new code not only uses PowerShell cmdlets but also takes into account the fact that PowerShell must ensure both files really exist before comparing the modification date of two files. Great!

The answer provided by ChatGPT contains a step-by-step explanation of the new script, which we haven’t reproduced here for space reasons. We’ve also had negative experiences with this type of translation task, regardless of the programming language: from time-to-time ChatGPT invents commands, cmdlets, or functions that don’t even exist.

1.1.6 Paid ChatGPT Offers

ChatGPT is basically free. If you’re willing to spend \$20 per month, ChatGPT Plus gives you access to the latest language model, more resources, and additional features. In the past (GPT 3.5 versus GPT 4.0), the difference in quality was quite noticeable. The paid version often provided better code and clearer descriptions, and it was much more up to date (due to training with newer material).

A major advantage of the Plus version is the ability to use specific GPTs. These are wizards that are optimized for specific tasks. For example, you can use the *Data Analyst* GPT to statistically evaluate and visualize data volumes. Simply drag and drop a file with the underlying data material into the chat history. The generated code gets executed immediately by ChatGPT, and you can view the result directly in the web browser.

You can also define your own GPTs with very little effort. An example of this follows in [Section 1.2](#). Specific GPTs can significantly speed up the completion of some tasks.

Companies can activate ChatGPT Team for \$25 per month per employee. The main difference compared to ChatGPT Plus is that the transmitted data isn't used for training ChatGPT. This is an advantage in terms of data protection, of course, but it doesn't change the fact that you still have to upload your code, your ideas and, ultimately, company secrets to the cloud to use ChatGPT. If you or your company have data protection concerns, you should consider running language models locally. We'll go into this topic in detail in the second part of the book (in particular, [Chapter 9](#)).

In our experiments, we've already had excellent experiences with the free ChatGPT version. The functions provided are great and absolutely sufficient for learning a programming language or for creating hobby applications.

But if you make a living from coding, \$25 per month for more up-to-date-ness and better-quality results is a good investment. However, you also pay for various additional features that aren't relevant for coding, such as imaging and audio/video functions. In this respect, the question arises as to whether a GitHub Copilot subscription ([Section 1.3](#)) might be the better investment.

Most of the ChatGPT results presented in this book were generated with version GPT-4o (4 *omni*). By the time you read this book, however, there will probably be newer versions of GPT that work even better.

1.1.7 Anthropic Claude

We've already referred to various ChatGPT alternatives (Google Gemini, Microsoft Copilot, etc.) in the introduction to this section. As far as software development is concerned, we became particularly fond of *Claude* from Anthropic (<https://claude.ai>) while we were working on this book.

Like ChatGPT, Claude can be used free of charge to a limited extent but requires a paid account for more intensive use. The web interface is very well designed and easy to use. In our tests, the quality of the results was at least on a par with those of ChatGPT, sometimes even better. As of December 2024, Claude was also characterized by a larger context window,

meaning it could process more user data and longer listings. Just give it a try!